

Project: AWS Auto Deployment Using Docker

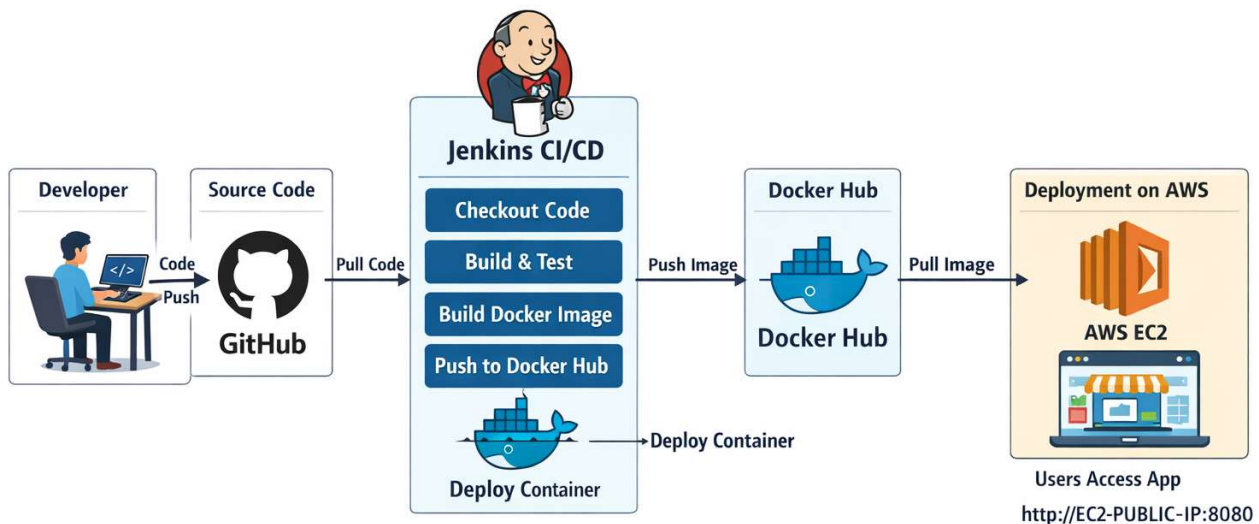
Definition:

AWS auto deployment using Docker is a process where applications are automatically built, containerized, and deployed in the cloud environment without manual intervention. In this approach, the source code is stored in GitHub and integrated with Jenkins to create a CI/CD pipeline. Whenever a developer pushes code changes, Jenkins automatically pulls the latest code, builds the application, and creates a container image using Docker. The Docker image is then pushed to Docker Hub, which acts as a storage repository for container images. Finally, the application container is automatically deployed and executed on an EC2 instance provided by Amazon Web Services. This automated deployment process ensures faster delivery, reduces manual errors, and provides a scalable and reliable environment for running applications in the cloud.

Scope of the Project:

- To develop an automated CI/CD pipeline for efficient application deployment.
- To manage source code using GitHub.
- To automate the build and deployment process using Jenkins.
- To containerize the application using Docker for consistent runtime environments.
- To store and manage Docker images in Docker Hub.
- To deploy the containerized application on cloud infrastructure provided by Amazon Web Services.
- To reduce manual deployment errors through automation.
- To enable faster and more reliable application delivery.
- To ensure scalability and portability of the application across environments.
- To demonstrate a real-world DevOps workflow for modern software development.

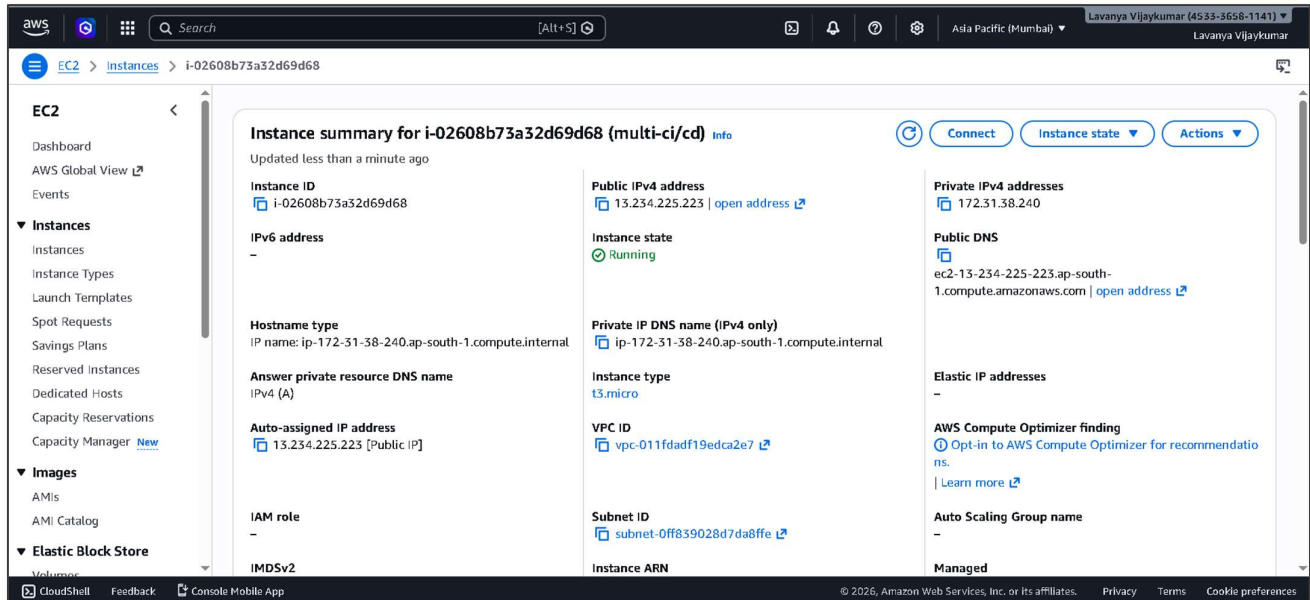
Architecture Diagram:



Project Implementation Steps:

STEP 1: Launch EC2 Instance

- Amazon Linux 2
- t3. micro
- Launch instance.

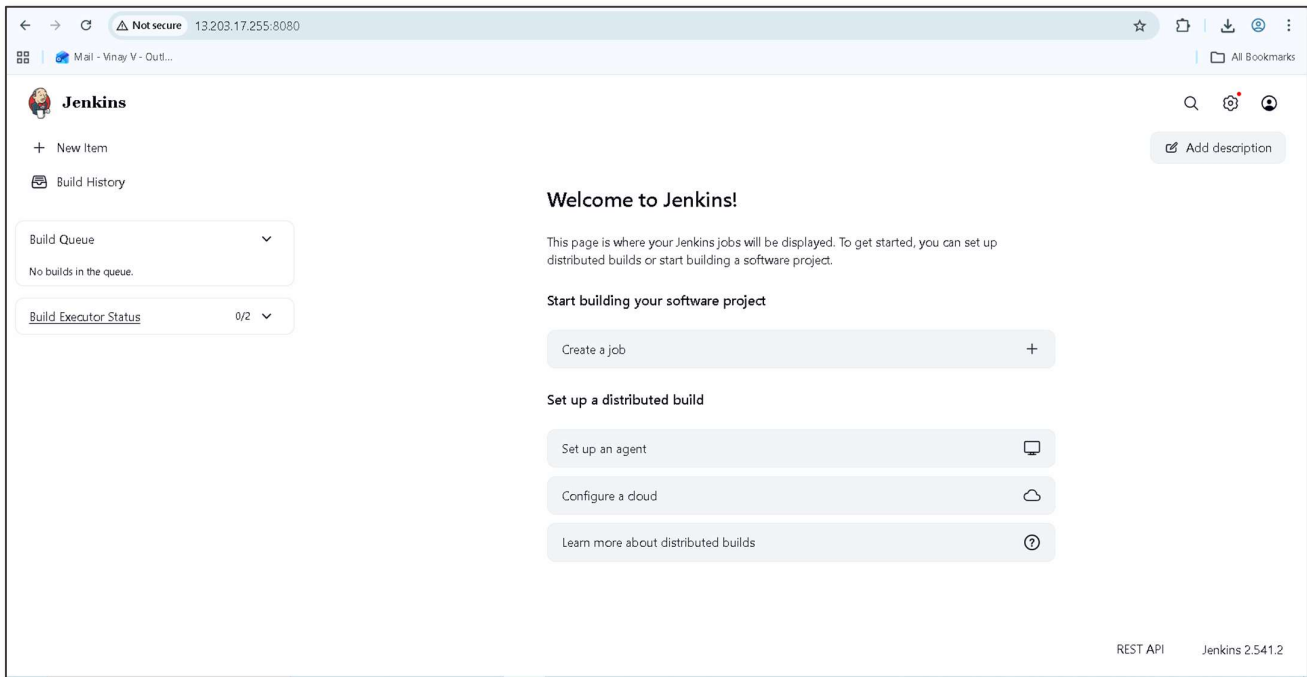


STEP 2: Install Java, Jenkins, docker, maven, git in ec2 instance

```
[ec2-user@ip-172-31-30-177 ~]$ java --version
openjdk 17.0.18 2026-01-20 LTS
OpenJDK Runtime Environment Corretto-17.0.18.9.1 (build 17.0.18+9-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.18.9.1 (build 17.0.18+9-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-30-177 ~]$ jenkins --version
2.541.2
[ec2-user@ip-172-31-30-177 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-30-177 ~]$ mvn --version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.1.163-186.299.amzn2023.x86_64", arch: "amd64", family: "unix"
[ec2-user@ip-172-31-30-177 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-30-177 ~]$
```

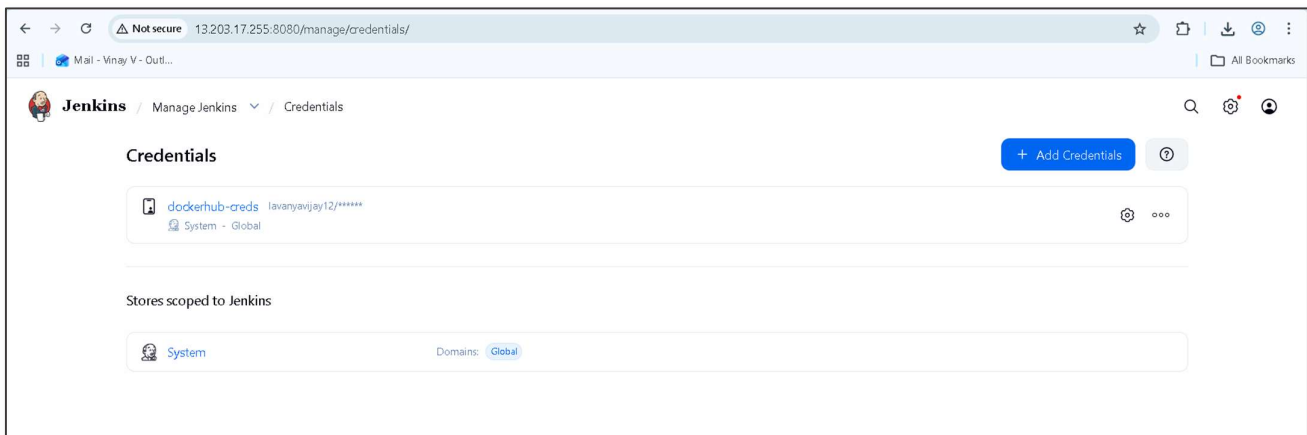
STEP 3: Add custom tcp with port of 8080 in security grp

- Type <http://ec2-ip:8080> URL in browser to access Jenkins dashboard
- Copy paste the InitialAdminPassword
- Install plugins and add username, password etc..



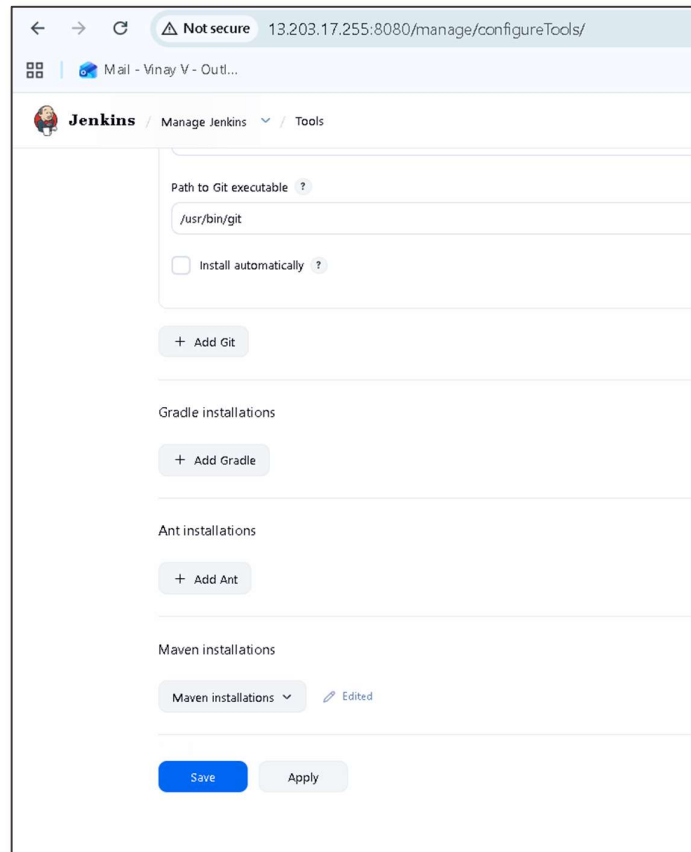
STEP 4: Configure DockerHub Credential

- Manage Jenkins → Credentials → Global → Add Credentials
- Select:
- Kind: Username with password
ID: dockerhub-creds
Username: your DockerHub username
Password: your DockerHub password

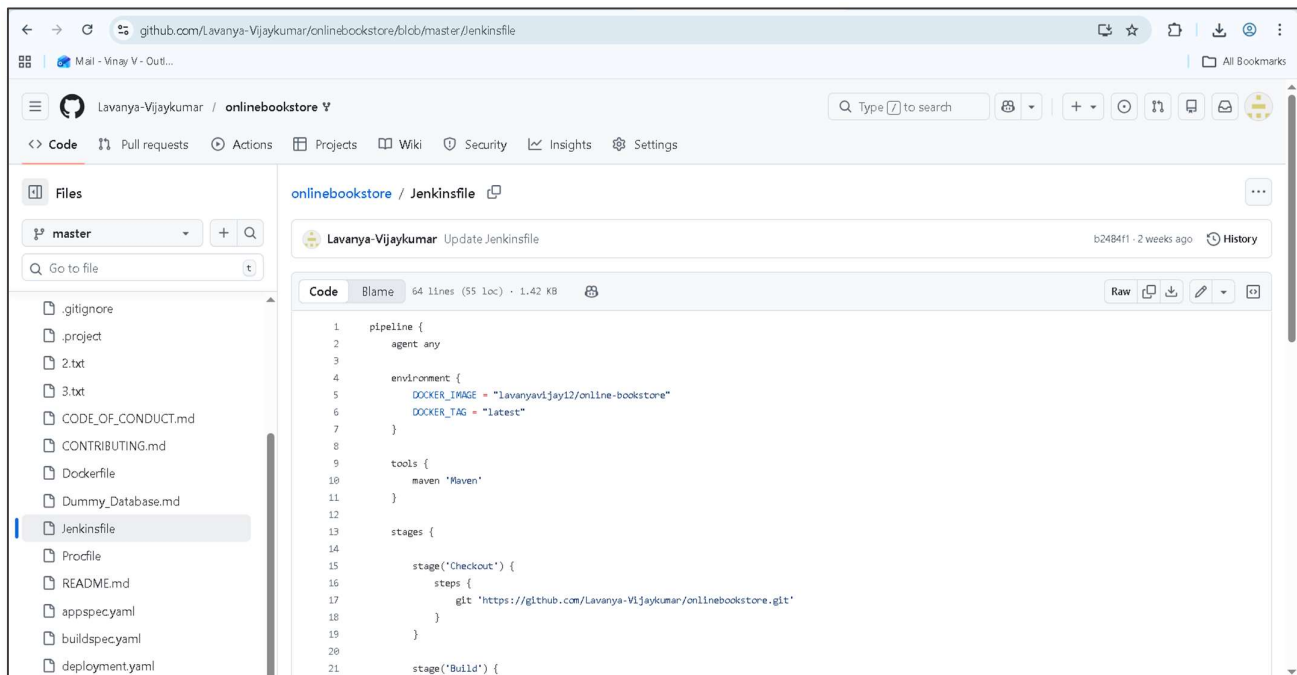


STEP 5: Configure Git and maven in Jenkins UI

- Manage Jenkins → Global Tool Configuration
- Scroll to Git
- Add:Name=Git
- Path to Git:/usr/bin/git
- Scroll to maven
- Add:Name=Maven
- Path to Git:/usr/share/maven
- Click Save

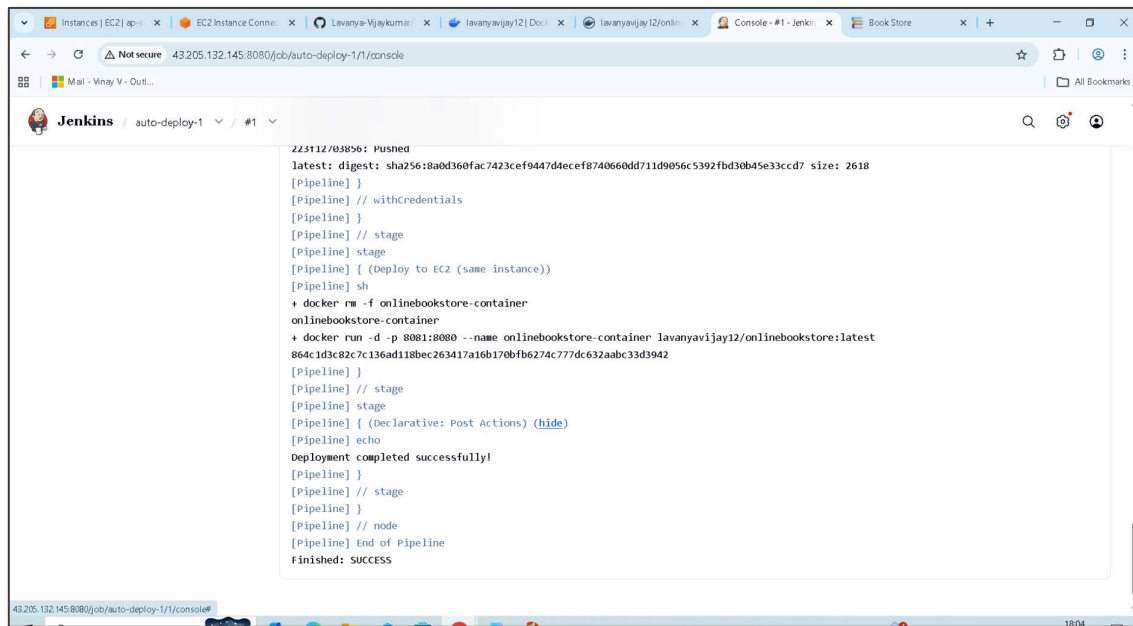
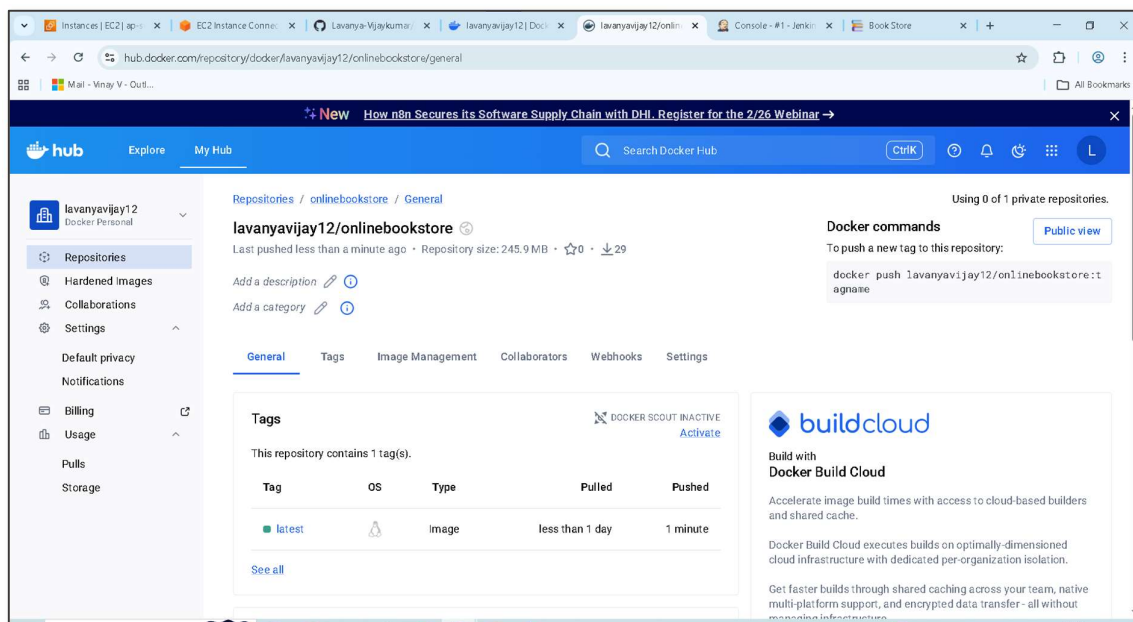


STEP 6: Add jenkinsfiles in git repository



STEP 7: Create Pipeline Job in Jenkins

- New Item
- Name: online-bookstore-pipeline
- Select Pipeline
- Pipeline → Definition: Pipeline script
- Add script in box
- Save
- Click: Build Now

**STEP 8: Output Build Docker image, Push to DockerHub, Run container in same EC2**

```

25-Feb-2026 12:13:23.180 INFO [main] org.apache.coyote.AbstractProtocol.init Initializing ProtocolHandler ["http-nio-8080"]
25-Feb-2026 12:13:23.209 INFO [main] org.apache.catalina.startup.Catalina.load Server initialization in [544] milliseconds
25-Feb-2026 12:13:23.266 INFO [main] org.apache.catalina.core.StandardEngine.startInternal Starting service [Catalina]
25-Feb-2026 12:13:23.287 INFO [main] org.apache.catalina.startup.HostConfig.deployWAR Deploying web application archive [/usr/local/tomcat/webapps/bookstore.war]
25-Feb-2026 12:13:23.910 INFO [main] org.apache.jasper.servlet.TldScanner.scanAll At least one JAR was scanned for TLDs yet contained no TLDs. Enable debug logging for this logger for a complete list of JARs that were scanned but no TLDs were found in them. Skipping unnneeded JARs during scanning can improve startup time and JSP compilation time.
25-Feb-2026 12:13:23.957 INFO [main] org.apache.catalina.startup.HostConfig.deployWAR Deployment of web application archive [/usr/local/tomcat/webapps/bookstore.war] has finished in [671] ms
25-Feb-2026 12:13:23.961 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-8080"]
25-Feb-2026 12:13:23.971 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in [759] milliseconds
[ec2-user@ip-172-31-17-34 ~]$ docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED            STATUS             PORTS               NAMES
864cd3c82c7        lavanyavijay12/onlinebookstore:latest "catalina.sh run"      32 seconds ago     Up 32 seconds      0.0.0.0:8081->8080/tcp, :::8081->8080/tcp onlinebookstore
[ec2-user@ip-172-31-17-34 ~]$ docker ps -a
CONTAINER ID        IMAGE               COMMAND                  CREATED            STATUS             PORTS               NAMES
864cd3c82c7        lavanyavijay12/onlinebookstore:latest "catalina.sh run"      36 seconds ago     Up 35 seconds      0.0.0.0:8081->8080/tcp, :::8081->8080/tcp onlinebookstore
[ec2-user@ip-172-31-17-34 ~]$ docker exec -it onlinebookstore-container bash
"docker exec" requires at least 2 arguments.
See "docker exec --help".

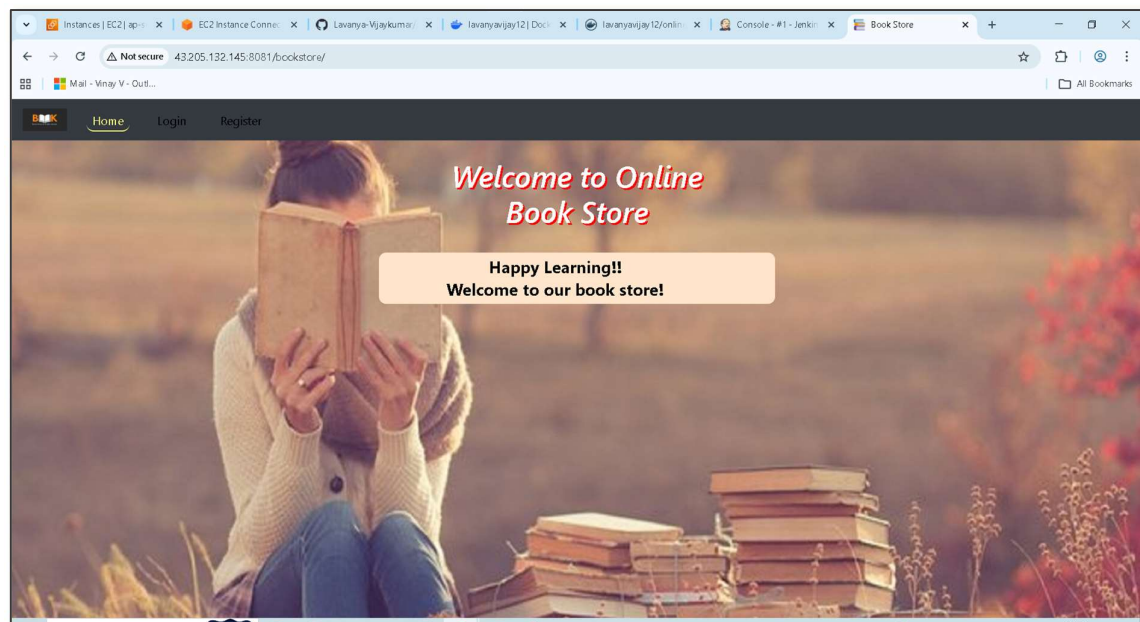
Usage: docker exec [OPTIONS] CONTAINER COMMAND [ARG...]

Execute a command in a running container
[ec2-user@ip-172-31-17-34 ~]$ docker exec -it onlinebookstore-container bash
Error response from daemon: No such container: onlinebookstore-container
[ec2-user@ip-172-31-17-34 ~]$ docker exec -it onlinebookstore-container bash
root@864cd3c82c7:/usr/local/tomcat# ls
bin          conf         filtered-KEYS  LICENSE      native-jni-lib  README.md    RUNNING.txt  upstream-KEYS  webapps.dist
BUILDING.txt  CONTRIBUITING.md  lib          NOTICE      RELEASE-NOTES   webapps
root@864cd3c82c7:/usr/local/tomcat#

```

STEP 9: Access Application

- Type URL http://YOUR_EC2_IP:8080/bookstore



Conclusion:

The CI/CD pipeline automated using Docker successfully demonstrates how modern DevOps practices can streamline the process of application development, integration, and deployment. By integrating GitHub for source code management, Jenkins for continuous integration and automation, and Docker for containerizing the application, the project ensures a consistent and reliable deployment process. The Docker images are stored in Docker Hub and deployed on cloud infrastructure provided by Amazon Web Services. This automated pipeline reduces manual effort, minimizes errors, improves deployment speed, and provides a scalable solution for modern application delivery. Overall, the project highlights the importance of CI/CD automation in achieving efficient and continuous software deployment in cloud environments.